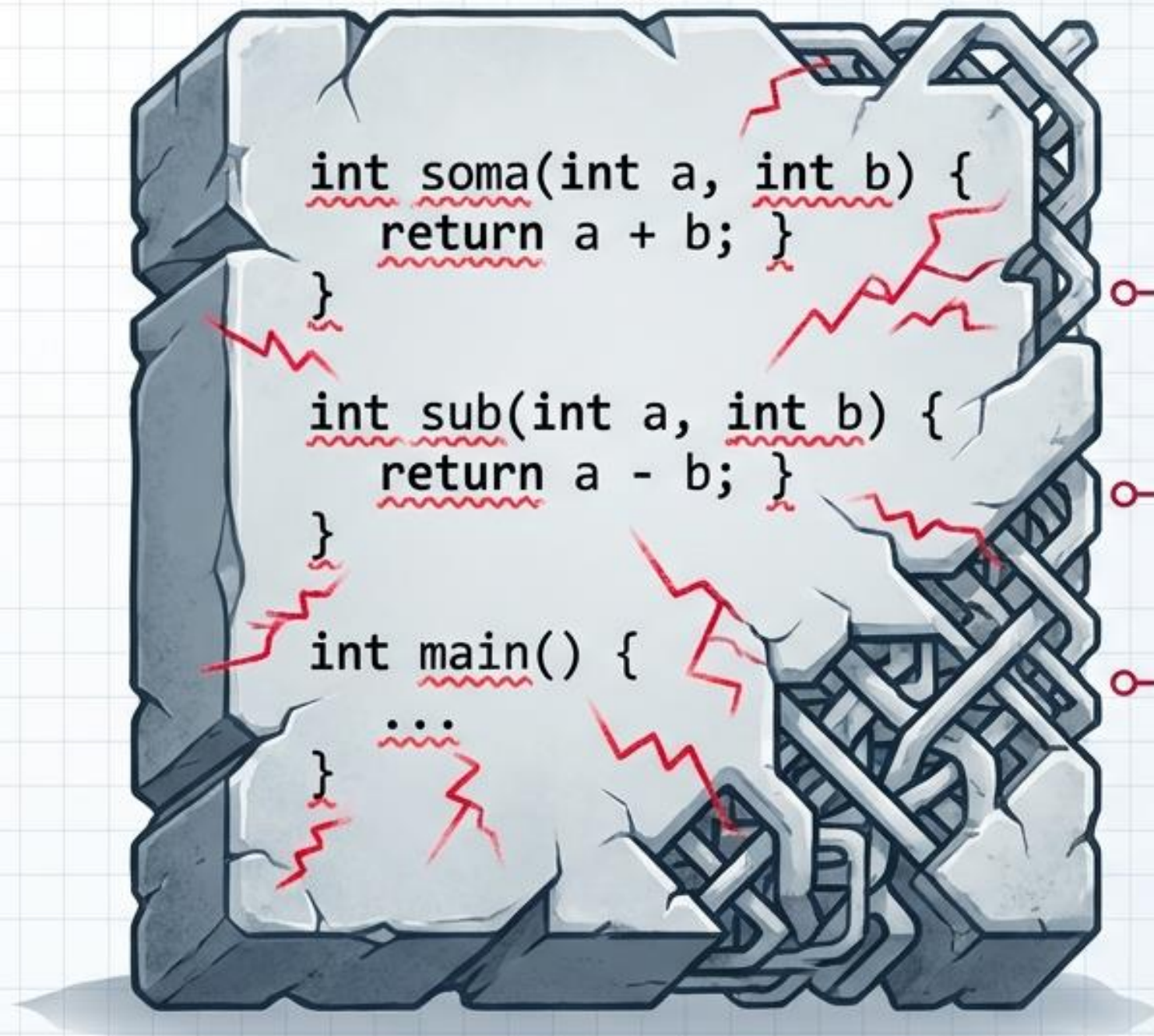


Arquitetura de Código em C: Do Monólito à Modularização

Um guia visual para transição de scripts simples para projetos multifile robustos.

A Síndrome do Arquivo Único



Crescimento Rápido: O código se torna impossível de navegar.



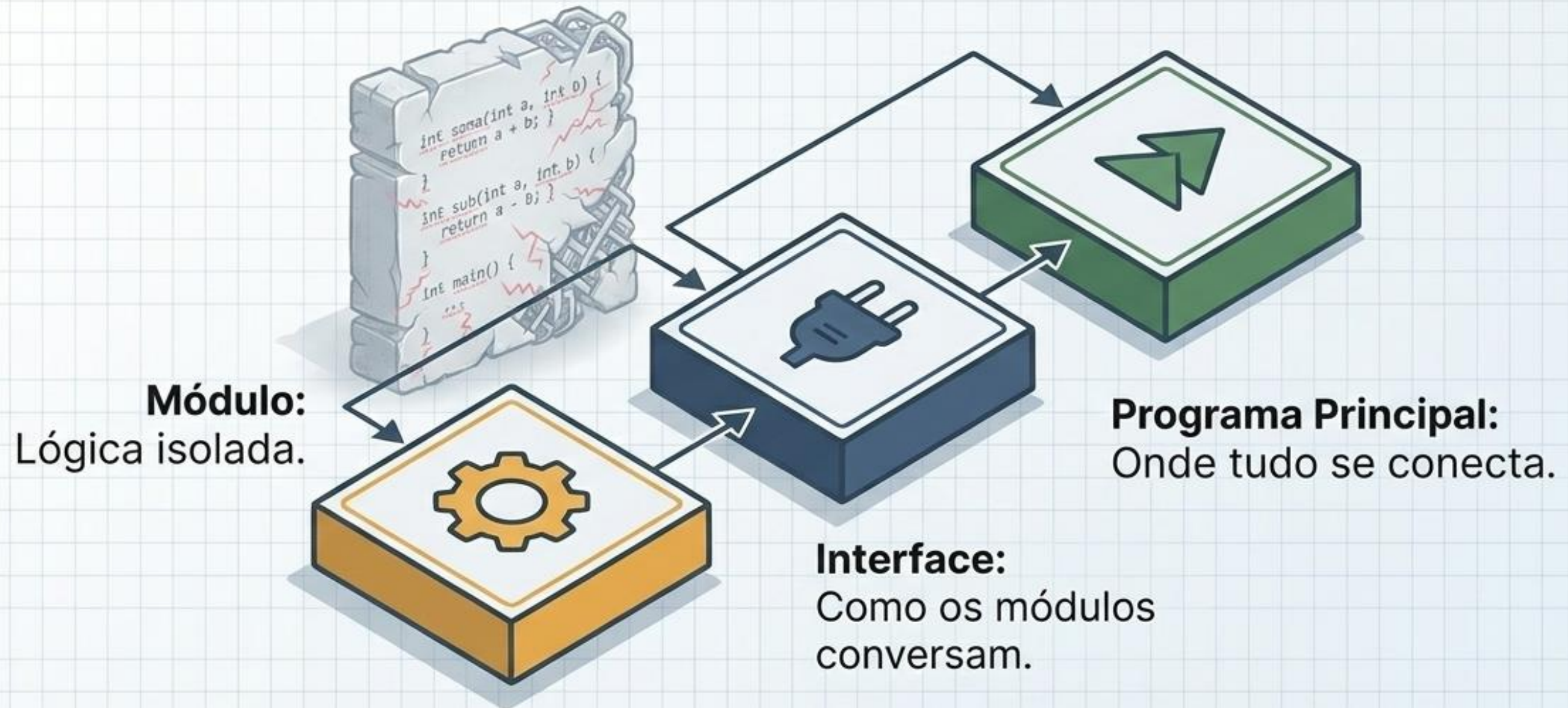
Manutenção Caótica: Alterar uma função arrisca quebrar todo o sistema.



Zero Reutilização: As lógicas de soma e sub estão presas a este arquivo.

A Solução: Engenharia de Módulos

Modularização consiste em dividir o programa em partes menores (módulos), cada uma com uma responsabilidade única e específica.



A Anatomia de um Módulo: Interface vs. Implementação

	.h (Header)	.c (Implementação)
Propósito:	Define a Interface	Executa a Lógica
Conteúdo:	Apenas Declarações <code>int soma(int a, int b);</code>	O Código Real <code>return a + b;</code>
A Analogia:	O Cardápio (O que o restaurante oferece) 	A Cozinha (Como o prato é feito) 
Usuário:	Outros módulos que precisam usar a função	O Compilador que vai montar o executável

A Mecânica da Diretiva #include

O **pré-processador** faz uma cópia literal do conteúdo do arquivo incluído e o cola no código fonte.

O Mecanismo

operacoes.h

```
/* operacoes.h */  
int soma(int a, int b);  
int subtracao(int a, int b);
```

Cópia literal

main.c

```
/* main.c */  
#include <stdio.h>  
#include "operacoes.h"  
#include "operacoes.h" // Esta linha será substituída  
  
int main() {  
    // O conteúdo de operacoes.h é inserido aqui  
    // ...  
    return 0;  
}
```

Bibliotecas Padrão (Sistema)

```
#include <stdio.h>
```

O compilador busca nos diretórios do sistema (Sintaxe com < >).

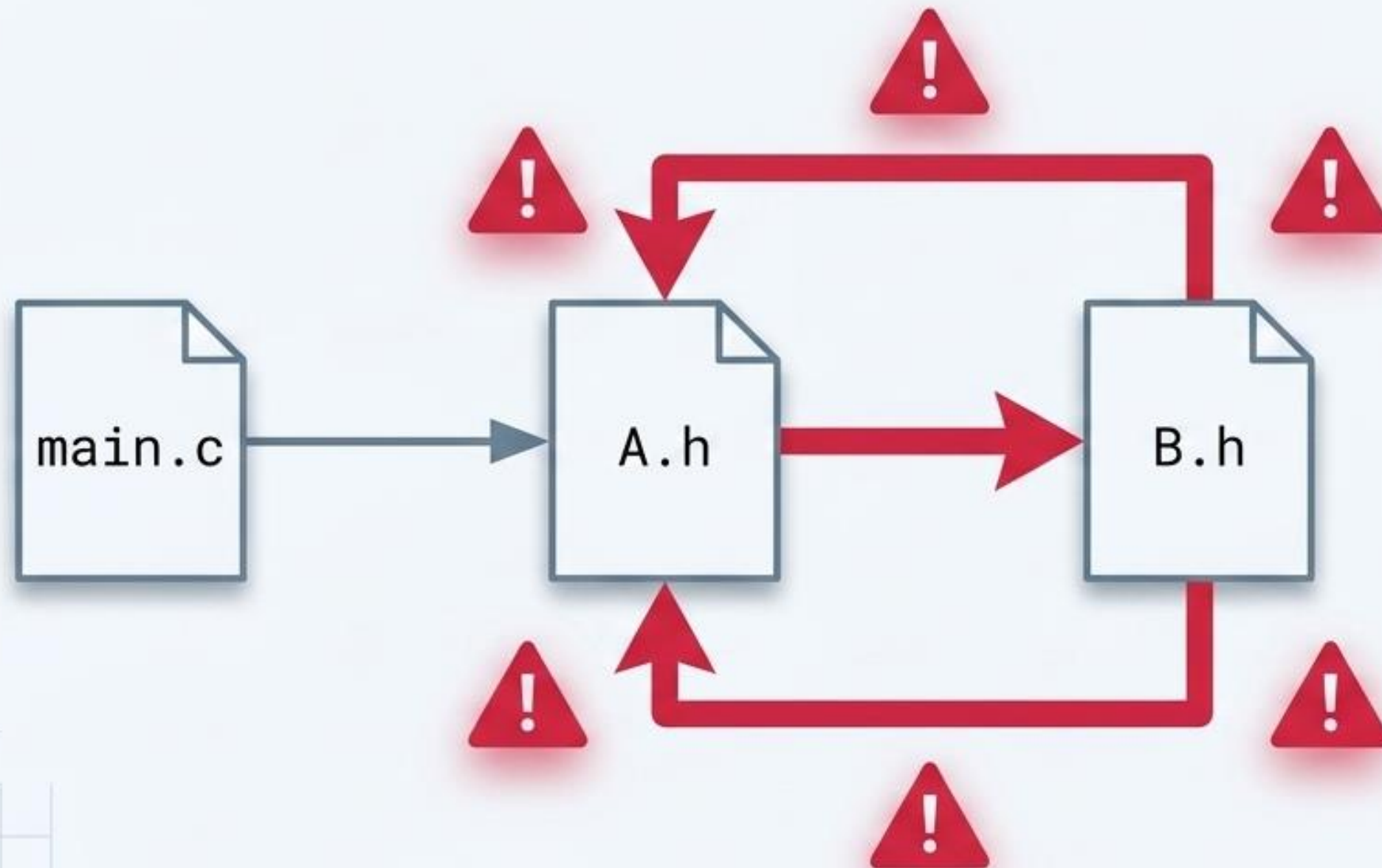
Bibliotecas do Usuário (Local)

```
#include "operacoes.h"
```

O compilador busca na pasta do projeto atual (Sintaxe com " ").

A Armadilha da Inclusão Múltipla

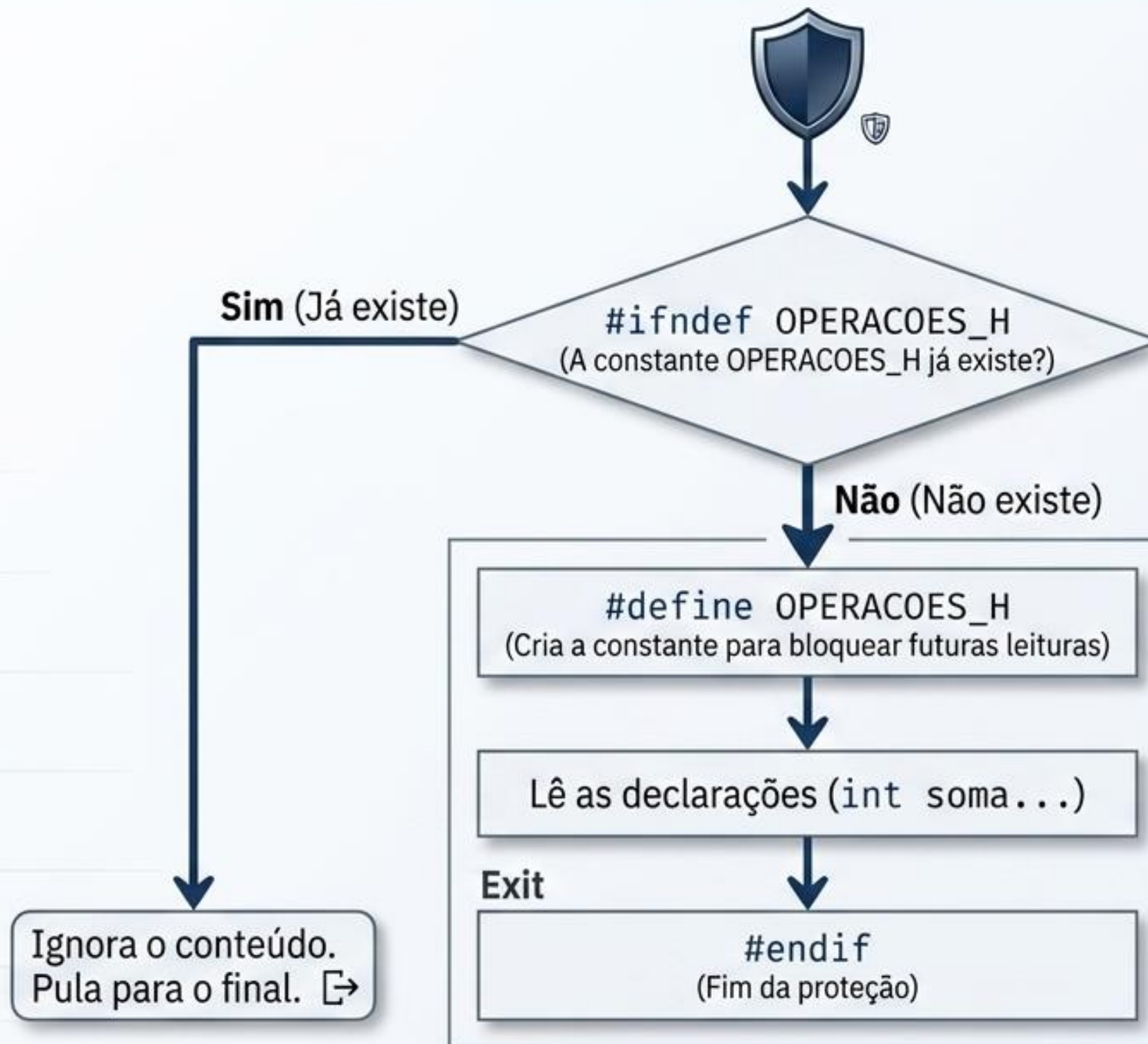
`main.c` inclui `A.h`. Mas `A.h` inclui `B.h`, que por sua vez inclui `A.h` novamente.



Sem proteção, o arquivo é incluído várias vezes, gerando:

- Redefinição de funções
- Declarações duplicadas de estruturas
- Conflitos de variáveis e erro de compilação!

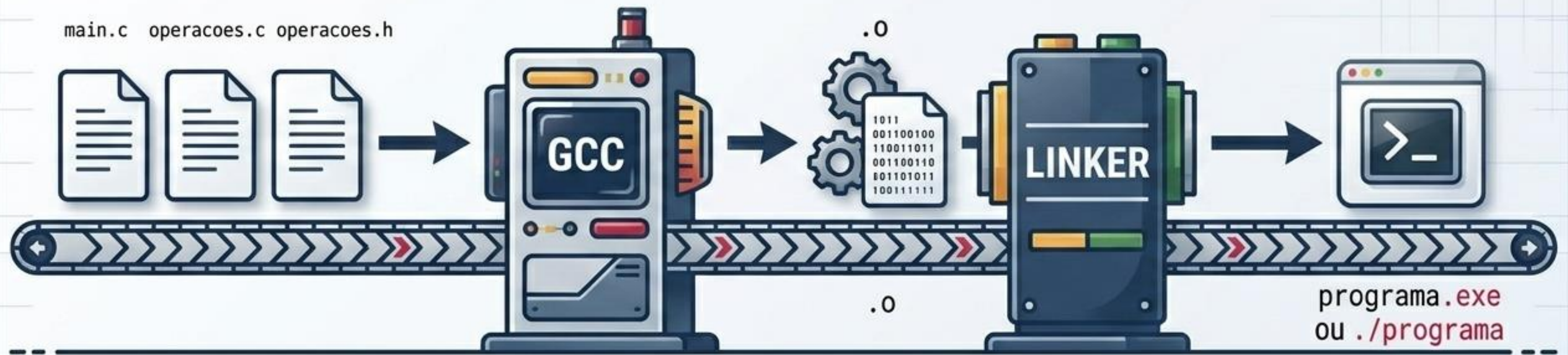
0 Escudo: Include Guards



Alternativa: `#pragma once`

Faz o mesmo com uma linha, mas não faz parte do padrão oficial do C e pode falhar em compiladores antigos. Prefira o padrão **`#ifndef`**.

O Ecossistema da Compilação



1. Código Fonte:
Arquivos .c e .h
(suas instruções).

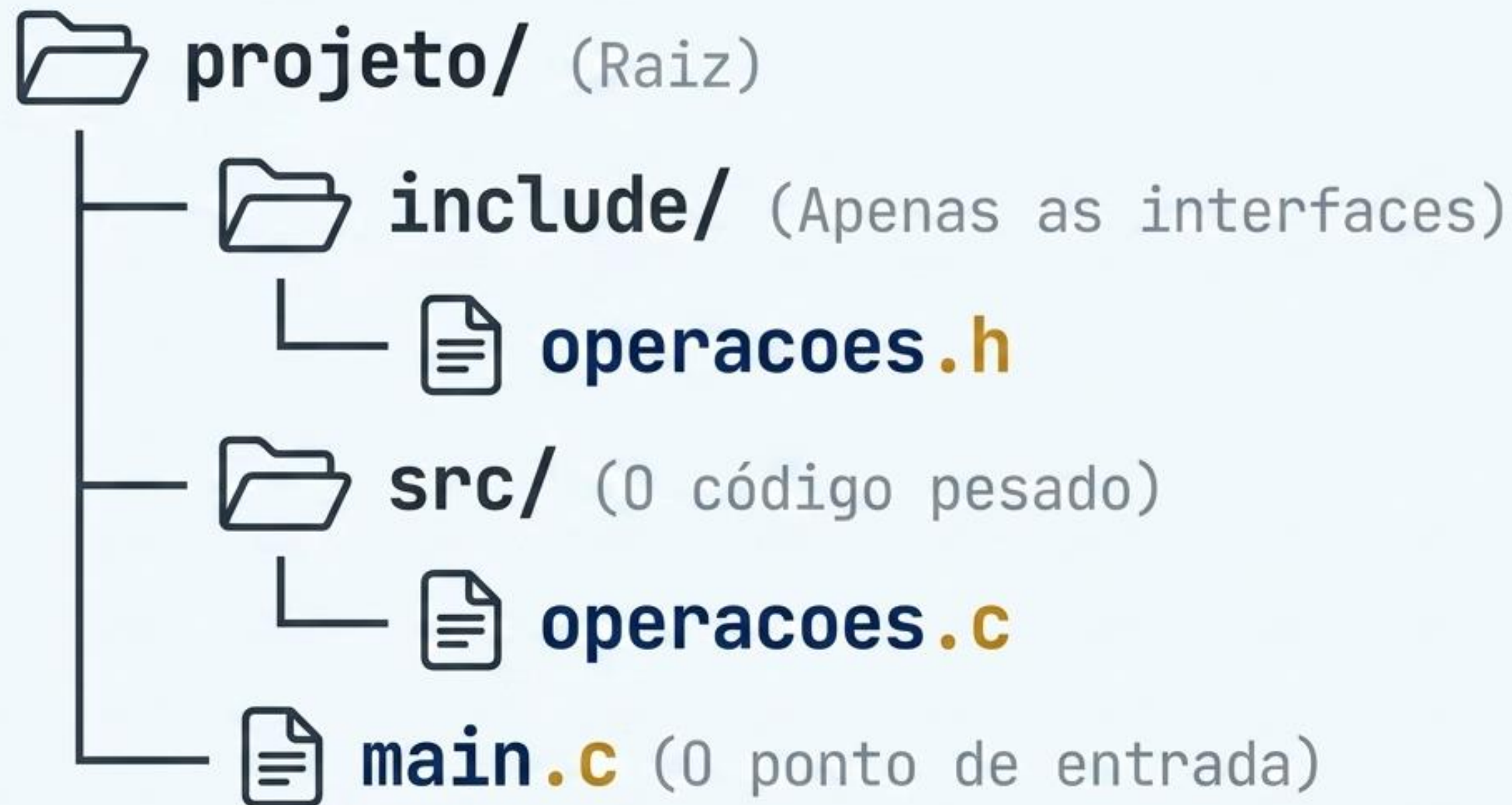
2. Pré-Processador & Compilador: Resolve os `#include` e transforma o texto em código de máquina.

3. O Linker (O Montador):
Une todos os arquivos objeto em um único sistema coeso.

4. Produto Final:
O Arquivo Executável
(programa.exe ou ./programa).

A Planta Baixa do Projeto

Organização profissional de diretórios.



Conectando as Peças (Estudo de Caso)

include/operacoes.h

```
#ifndef OPERACOES_H
#define OPERACOES_H

int soma(int a, int b);
#endif
```

src/operacoes.c

```
#include "operacoes.h"

int soma(int a, int b) { return a + b; }
```

main.c

```
#include <stdio.h>
#include "operacoes.h"

int main() { printf("Soma: %d\n", soma(2,3));
            return 0;
}
```

Acionando a Linha de Montagem

```
gcc main.c src/operacoes.c -Iinclude -o programa
```

O compilador
(A ferramenta).

Os arquivos de
origem
(Nunca compile o .h
diretamente!).

A flag de diretório
(Diz ao GCC: 'Procure
os arquivos .h na
pasta include').

O nome da saída
(Como o executável
final será chamado).

Linux/Mac: `./programa` | Windows: `programa.exe`

Saída Esperada: Soma: 5

Padrões de Excelência (Checklist do Desenvolvedor)

Lista de Verificação do Piloto

- ✓ **Proteção Sempre:** Utilize Include Guards em absolutamente todos os arquivos .h.
- ✓ **Separação Estrita:** Nunca implemente lógica (corpo de funções) dentro de um .h, apenas as declare.
- ✓ **Organização Lógica:** Mantenha a estrutura de pastas separada (src, include) e agrupe lógicas por tema.
- ✓ **Documentação:** Comente a finalidade das funções diretamente nas declarações do arquivo .h.

Dica Pro (Modo Paranoico Ativado)

Sempre compile com os avisos ativados para capturar erros invisíveis:

```
gcc -Wall -Wextra main.c  
src/operacoes.c -Iinclude  
-o programa
```